

Triangle Algorithm

1- First Algorithm (Non-Recursive)

- Sort the zero-indexed array by Insertion Sort.
- Make Loop for Triangle Check Condition.

1.1: Pseudo Code:

ALGORITHM isTriangle(A[0..N-1])

// Step 1: Insertion Sort

```
for i ← 1 to N-1 do
    key ← A[i]
    j ← i-1
    while j >= 0 AND A[j] > key do
        A[j+1] ← A[j]
        j ← j - 1
    end while
    A[j+1] ← key
endfor
```

// Step 2: Triangle Check

```
for i ← 0 to N-3 do
    if A[i] + A[i+1] > A[i+2] then
        return 1
    end if
end for
return 0
```

1.2: Code implementation in c :

```
#include <stdio.h>

int isTriangle(int A[], int n){
    for(int i = 1; i < n; i++){
        int key = A[i];
        int j = i - 1;
        while(j >= 0 && A[j] > key){
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
    for(int i = 0; i < n - 2; i++){
        if(A[i] > 0 && A[i] + A[i + 1] > A[i + 2]
    ){
        return 1;
    }
    }
    return 0;
}

int main(){
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int A[n];
    printf("Enter elements: \n");

    for (int i = 0; i < n; i++){
        scanf("%d", &A[i]);
    }

    if(isTriangle(A, n)){
        return 1;
    } else {
        return 0;
    }
    return 0;
}
```

1.3 Mathematical Analysis of the Non-Recursive Algorithm

The non-recursive approach consists of two sequential operations: sorting the array via Insertion Sort, and subsequently scanning the array via a linear loop to verify the triangle condition. We analyze the Best, Average, and Worst-case scenarios for each operation.

Step 1: Insertion Sort Analysis

Insertion Sort utilizes two nested loops. The outer loop runs from $i = 1$ to $N - 1$, while

the inner `while` loop runs backward depending on whether the current element is greater than the `key`.

- **Worst-Case Scenario:** Occurs when the array is sorted in reverse order. The inner loop shifts every preceding element.

- **Summation:**

$$C(N) = \sum_{i=1}^{N-1} i$$

- **Resolution:** Using the arithmetic series sum formula, we get $\frac{N(N-1)}{2}$.

- **Complexity:** Dropping constants yields $O(N^2)$

- **Best-Case Scenario:** Occurs when the array is already perfectly sorted. The inner loop condition fails immediately, resulting in exactly one comparison per outer iteration.

- **Summation:**

$$C(N) = \sum_{i=1}^{N-1} 1$$

- **Resolution:** $(N - 1) - 1 + 1 = N - 1$.

- **Complexity:** Dropping constants yields $O(N)$

- **Average-Case Scenario:** Occurs with randomly ordered arrays. On average, the inner loop shifts half of the preceding elements, running $i/2$ times.

- **Summation:**

$$C(N) = \sum_{i=1}^{N-1} \frac{i}{2}$$

- **Resolution:** $\frac{1}{2} \frac{N(N-1)}{2} = \frac{N^2-N}{4}$
- **Complexity:** $O(N^2)$

Step 2: Linear Triangle Check Analysis

A single `for` loop iterates from 0 to $N - 3$ to check the condition $A[i] + A[i+1] > A[i+2]$.

- **Worst/Average-Case Scenario:** The loop reaches the end of the array without finding a triangle (or finds it at the very end).

- **Summation:**

$$C(N) = \sum_{i=0}^{N-3} 1$$

- **Resolution:** $(N - 3) - 0 + 1 = N - 2$.
- **Complexity:** $O(N)$

- **Best-Case Scenario:** The first three elements form a valid triangle, returning immediately.

- **Complexity:** $O(1)$

Total Time Complexity (Non-Recursive)

By applying the dominant term rule:

- **Worst/Average Case:** $O(N^2)_{[Sort]} + O(N)_{[Check]} = O(N^2)$
- **Best Case:** $O(N)_{[Sort]} + O(1)_{[Check]} = O(N)$

2- Second Algorithm (Recursive)

2.1:Pseudo Code:

ALGORITHM merge(A[0..N-1], p, q, r)

$i \leftarrow p, j \leftarrow q+1, k \leftarrow 0$

create temp array B

while $i \leq q$ **AND** $j \leq r$ **do**

if $A[i] \leq A[j]$ **then**

$B[k] \leftarrow A[i], i \leftarrow i+1$

else

$B[k] \leftarrow A[j], j \leftarrow j+1$

end if

$k \leftarrow k+1$

end while

copy remaining elements of left or right into B

copy B back into A[p..r]

ALGORITHM mergeSort(A[0...N-1], p, r)

if $p < r$ **then**

$q \leftarrow \text{FLOOR}[(p + r) / 2]$

 mergeSort(A, p, q)

 mergeSort(A, q+1, r)

 merge(A, p, q, r)

end if

ALGORITHM isTriangleCheck(A[0..N-1], i, N)

if $i \geq N-2$ **then**

return 0

end if

if $A[i] + A[i+1] > A[i+2]$ **then**

return 1

end if

return isTriangleCheck(A, i+1, N)

ALGORITHM isTriangle(A[0...N-1], N)

 mergeSort(A,0, N-1)

return isTriangleCheck(A,0, N)

2.2: Code implementation in c :

```
#include <stdio.h>

// Merge function
void merge(int A[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = A[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = A[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            A[k] = L[i];
            i++;
        } else {
            A[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { A[k] = L[i]; i++; k++; }
    while (j < n2) { A[k] = R[j]; j++; k++; }
}

// Merge Sort (Recursive)
void mergeSort(int A[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(A, left, mid);
        mergeSort(A, mid + 1, right);
        merge(A, left, mid, right);
    }
}

// Recursive Triangle Check
int isTriangleCheck(int A[], int i, int N) {
    if (i >= N - 2)
        return 0;
    if (A[i] + A[i + 1] > A[i + 2])
        return 1;
    return isTriangleCheck(A, i + 1, N);
}

// Main Triangle Function
int isTriangle(int A[], int N) {
    mergeSort(A, 0, N - 1);
    return isTriangleCheck(A, 0, N);
}

int main() {
    int N;
    printf("Enter number of elements: ");
    scanf("%d", &N);
    int A[N];
    printf("Enter elements: ");
    for (int i = 0; i < N; i++) {
        scanf("%d", &A[i]);
    }
    if (isTriangle(A, N))
        printf("Output: 1 (Triangle exists)\n");
    else
        printf("Output: 0 (No triangle)\n");

    return 0;
}
```

2.3: Mathematical Analysis of the Recursive Algorithm

The recursive approach utilizes a Divide and Conquer strategy by sorting the array via Merge Sort, followed by a Decrease and Conquer recursive function to check for the triangle condition.

Step 1: Merge Sort Analysis

Merge Sort divides the array into halves until the base case is reached, then merges them in linear time. Its behavior remains consistent regardless of the initial array order.

- **Recurrence Relation:**

$$T(N) = 2T(N/2) + \Theta(N)$$

- **Solving via Master Method:**

- Identify variables: $a = 2$, $b = 2$, and $f(N) = N$.
- Calculate the criterion: $N^{\log_b a} = N^{\log_2 2} = N^1 = N$.
- Since $f(N) = \Theta(N^{\log_b a})$ (both are N), this falls exactly under **Case 2** of the Master Theorem.
- Resolution: We multiply by $\log N$, resulting in $T(N) = \Theta(N \log N)$.
- **Complexity:** The time complexity is $\Theta(N \log N)$ strictly for the Best, Average, and Worst cases.

Step 2: Recursive Triangle Check Analysis

The recursive check function processes the array sequentially, reducing the problem size by 1 with each recursive call, doing constant c work (comparisons) per call.

- **Worst/Average-Case Scenario (Recursion to the end):**

- **Recurrence Relation:**

$$T(N) = T(N - 1) + c$$

- **Solving via Iteration Method:** Expanding the recurrence:

$$T(N) = (T(N - 2) + c) + c = T(N - 2) + 2c$$

$$T(N) = (T(N - 3) + c) + 2c = T(N - 3) + 3c$$

General pattern after k steps:

$$T(N) = T(N - k) + kc$$

- **Base Case:** The recursion terminates when only 2 elements are left ($N-k=2$), yielding $k = N - 2$.
- **Resolution:** Substituting k back into the pattern:

$$T(N) = T(2) + (N - 2)c$$

Assuming $T(2)$ is a constant c_0 :

$$T(N) = cN - 2c + c_0$$

- **Complexity:** Dropping lower-order terms yields $\Theta(N)$
- **Best-Case Scenario:** The recursion finds a valid triangle on its very first call and terminates immediately.
- **Complexity:** $O(1)$

Total Time Complexity (Recursive)

Combining both steps and retaining the dominant term:

- **Worst/Average Case:** $\Theta(N \log N)_{[Sort]} + \Theta(N)_{[Check]} = \Theta(N \log N)$
- **Best Case:** $\Theta(N \log N)_{[Sort]} + O(1)_{[Check]} = \Theta(N \log N)$

Comparison:

The non-recursive solution uses Insertion Sort to sort the array and a simple loop to check the triangle condition. The recursive solution uses Merge Sort for sorting and a recursive function for the triangle check.

	Non-Recursive	Recursive
Sorting	Insertion Sort	Merge Sort
Triangle Check	Loop	Recursive function
Sort Complexity	$O(n^2)$	$O(n \log n)$
Check Complexity	$O(n)$	$O(n)$
Total Complexity	$O(n^2)$	$O(n \log n)$
Better for large input	No	Yes
Simpler code	Yes	No

Conclusion:

The recursive solution is more efficient for large inputs due to Merge Sort's $O(n \log n)$ complexity compared to Insertion Sort's $O(n^2)$. However, the non-recursive solution is simpler to implement and understand.